Mahboob Sir

16.6.26

CSE 301: Database management

Ref book

Database System Concept → Henry 7th edit 4th

⇒Must read book.

Chp - 1

Database এর concept গুলোতে set theory থেকে

Data: relevant unprocessed information.

Inconsistency - একই জায়গায় একই info of same person

Constraint: Banking এর savings acc

Exam ⇒ Drawbacks of using file system

⇒ concurrent access (একাধিক user একসাথে access), redundancy, data isolation, inconsistency, constraint, integrity এগুলো বুঝাতে হবে with example

⇒ এগুলোকিছু database এ করা easy not in file sys.

Data Models

⇒ Relational model: Tabular orientation

⇒ Entity-Relationship data model: design এ use হয়, pictorial view

Schema - design of the database

Instance: গতানুগতিক data, যে নির্দিষ্ট database এ access করার জন্য data

Exam ⇒ Physical level abstract

⇒ What is abstract

⇒ Difference bet schema and instance

What is database?

⇒ A collection of related information stored in an organized way

What is DBMS?

⇒ A collection of interrelated data and a set of programs used to access that data

⇒ DBMS is a software that helps us.
Store data, Find data, Update data, Delete data, Protect Data.

⇒ Example: MySQL, Oracle Database, Microsoft SQL Server, PostgreSQL

Why do we need DBMS

Without DBMS:

⇒ Information becomes messy

⇒ Data gets duplicated

⇒ Data gets lost

⇒ Searching becomes slow

Database Applications

⇒ Banking

Stores

1) Customer information, 2) Accounts, 3) Loans, 4) Transactions

Example: When we withdraw money from bank, database updates the balance

⇒ Airlines

Stores

1) Passenger information, 2) Flight schedules, 3) Reservations

Example: When we book a ticket online, data goes into the online database

⇒ Universities

Stores

1) Student records, 2) Courses, 3) Results, 4) Attendance

⇒ Sales

Stores

1) Products, 2) Customers, 3) Orders

⇒ Manufacturing

1) Inventory, 2) Orders, 3) Product info

Human resources

1) Employee records
2) Salaries
3) Payroll

Before DBMS existed, they used file systems:

Example: customer.txt, loan.txt

Each file stored info separately. This caused many problems like:

1) Data Redundancy

same data stored multiple times

Example: Customer Rahim's address appears in customer.txt, loan.txt

The same address is repeated

⇒ It results in wastage of storage

⇒ Hard to maintain data

2) Data Inconsistency

Ex: Rahim changes his address, one file becomes Dhaka, another file still contains Gazipur, which is correct nobody knows

This is data inconsistency

3) Difficulty in accessing data

Ex: Suppose a bank manager asks to show all customers living in a certain postal code, there may be no program for that

⇒ Very slow and inconvenient

4) Data isolation

Ex: data is stored in different files customer.xls, loan.txt, different formats

⇒ Combining data becomes difficult

5) Data Integrity

Integrity means data must remain ~~same~~/correct

Exam: A bank acc should never have balance = -500

This rule must be enforced ⇒ In file systems, it is difficult to maintain such rules

6) Atomicity problem

Ex: Transfer 500 Tk from acc-A to acc-B

Step 1	Step 2
Remove 500 from A	Add 500 to B

Suppose electricity goes goes after step 1

A lost 500, B received nothing, money disappears

DBMS prevent it

7) Concurrent access problem

Concurrent means multiple users accessing data at the same time

Ex: Acc balance: 1000, Two people withdraw money simultaneously

User 1 - Withdraw 500, User - 2: Withdraw 600

If DBMS does not control this, incorrect balances can occur

8) Security problem

Not everyone should see all info

Ex: Employee should not see CEO salary.

DBMS provides password, authentication, permission

Simplifies database usage and improves security

Data abstraction | DBMS hides storage complexity thru data abstraction

databases are very complex internally, to make them easy to use, DBMS hides complexity This is called data abstraction

3 levels of abstraction

1) Physical level - Lowest level

Data on disk → stored in files, blocks, pages → Low level binary data

⇒ Describes how data is stored internally

Ex: Disk blocks, files, indexes

Ordinary users never see this

2) Logical level - middle level ⇒ Table, relationships bet" tables, constraints and rules

⇒ Describes what data is stored, relationships bet" data

Ex: Customer
Name
Street
City

⇒ DBA usually works at this level

3) View level - Highest level

⇒ Shows only necessary info

Ex: Teacher sees: Student Name, Marks | Student sees: Student name, Marks hidden This is called a view

⇒ backed

2 types → Logical and Physical

Physical → database ঘর structure

Logical

Schema → database design

It is like a building blueprint

Ex. Customer
- ID
- Name
- City

This structure is schema. It changes very rarely

Instance

→ Actual data currently stored

ID	Name
101	Rahim

If data is changed then instance changes but schema remains the same

Physical data independence → The ability to modify the physical schema without changing the logical schema

Suppose the database changes how data is stored on disk, applications still work normally. This ability is called Physical Data Independence

⇒ Bank এ branch খুলে তাহলে database এ change করে update করে নেয়।

Data Models

⇒ A data model is a way to describe data, relationships, constraints

Entity Relationship (ER) model

⇒ Used for database design

Entity: an obj that we can distinguish from others it can be physically distinguishable or not, abstract in nature

Entity - an object in the real world

Ex - student, customer, account

Attributes - properties of entities

Ex - Customer
ID, Name, city These are attributes

Relationship - connection bet entities

Ex - Customer owns Accounts. This connection is a relationship

Relational model

⇒ most popular database model

⇒ stores data in tables

Ex

ID	Name	City
101	Rahim	Dhaka

⇒ Rows = Records

⇒ Columns = Attributes

Mahboob Sir

23.6.26

DL is divided into 2 types

DDL

⇒ data কে define করে
⇒ create tables
⇒ specify constraints
↓
কোন column এর type
কেমন, char নাকি int

Ex - create table
alter (column বাড়ানো কমানো)
drop (delete table)

⇒ Drop করলে table এ structure এ related instruct" থাকে
⇒ Alter, Drop এসব structure এ পরিবর্তন করতে

DML

⇒ data কে manipulate করা
⇒ data গুলো efficiently বের করে নিয়ে আসার (retrieve)

Ex
Select (data choose করে আনা, retrieve)
Insert
Update
Delete (data কে delete করা)
entire row কে delete করি

⇒ data না থাকলে no row
⇒ Frequently used command
⇒ Used in instance

Data dictionary

একটা table কে যখন create করি তখন by default আমের সাথে কিছু জিনিস create হয়ে একটা data dictionary (Compiler auto create করে)

⇒ Data dictionary contains metadata

Metadata - data about data

Book এ index file থাকে যেন efficiently data খুঁজে পায়

Excer ☆☆ What is primary key? → পরে পড়ব

Procedural DML - কী data চাই এবং কীভাবে
Non " " - কোন data চাই

Storage manager

Transact manager → recovery: একটা account থেকে taka minus হলো অন্য acc এ ঢুকতে পারলনা, তখন আবার first acc এ টাকা ঢুকে যাবে। This is recovery

Buffermanager → কাজ করার আগেই data টাকে নিয়ে আসে, to make it faster and efficient

⇒ temporary জায়গায় রাখবে, আর parallelly কাজ করতে থাকবে

Query processing

1) Optimizat - fastest way তে data টা কে retrieve করার জন্য

⇒ Statistics about data is called metadata

Transact management

ACID properties

Exercise থেকে 3 quest হয়

Must read book

Mahboob Sir

25.6.26

Chp-6: Database Design Using E-R model

☆☆ ER model V.V.I

This is from 4th edition

30.6.26

Mahboob Sir

Entity Sets → similar pattern এর ডিফারেন্ট একসাথে থাকা

→ Tentative design টা করে রেখার পর implementation যেতে পারবেনা, কারণ client দের requirements vary করে

☆ What is entity-relationship model

⇒ Diff obj (entity) create করে তাদের connect করে রেখে in database

Domain → particular attribute এর permissible value

Attribute types

⇒ Single valued and multi-valued attributes

Student id — multiple phone number

⇒ Derived attribute - কোনো কোনো attribute কে ডাইরেক্টলি রেখে না, extract করা লাগে -

Null value (Two situations)

⇒ যার কোনো phone number নেই (It doesn't exist)

⇒ phone no আছে, tracking এর (It exists but not given) জন্য রেখেনি

Relationship

⇒ একটা entity set এর সাথে আরেকটা entity set connect করবে কীভাবে

n ≥ 2 entities → দুইটা বা তার বেশি না হলে connect হবে না

Descriptive attribute

Glorifies the relation

☆ What is degree of relationship?

⇒ একটা entity র সাথে আরেকটা entity র একটাই relation → binary relationship

⇒ Binary = 2 (This is the degree of relationship)

⇒ কয়টা entity কে connect করছে সেটা degree

Mapping cardinalities

⇒ One to one → একটা customer একটা acc ই খুলতে পারে in Bank

⇒ One to many → Bank যদি বলে একজন cust একের বেশি acc খুলতে পারে

⇒ Many to one → Bank যদি বলে আপনে দুজন একটা acc (joint acc)

⇒ Many to many → Bank যদি বলে more than 1 acc হবে, joint acc ও হতে পারে

student - teacher

1) a - b, c - d

one to one

2) $a_3 \to c_4$, $a_3 \to b$, $a_3 \to c$, $a_2 \to b_3$, $a_2 \to c_3$

one to many

one to one এ exist করতে পারে এভাবে

3) $a_1, a_2 \to b_2$; $a_3 \to b_3$

many to one

4) $a_1 \to b_1$, $a_2 \to b_2$

many to many

Follow slide for E-R diagram (book ≠ অনুসরণ করা)

E-R diagram

- Rectangle → entity sets
- Ellipse → characteristics of obj
- Underline করা → primary key
- Diamond → relationship
- Links → connect"
- Double ellipse → multiple value
- Dashed " → derived attribute
  eg- age, cgpa
  → marks input দিলে cgpa generate হয়

Mid এ design আসবে

CT syllabus upto Roles Chp-6

Roles

2.7.26

Entity connecting to itself

manager একজন (attribute) workers অনেকজন

Worker, manager দুইজনেই Employee
→ Role mention করতে হয় এই case এ

Cardinality constraints

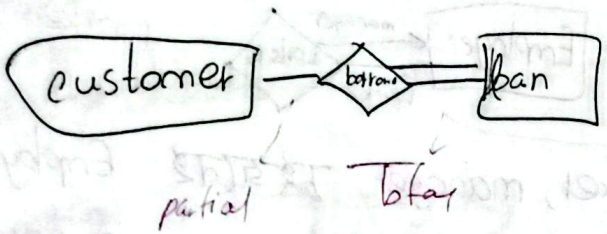
one to one হলে arrow
'→' signifies 'to one' or 'one to'
'—' " many

Total participation

Indicated by double line
সব customer loan নেই না. All customers don't participate in loan. But there can't be any loan without customer

Loan এর participat টা total

Customer " " " participat



V.V.I Keys ☆☆ Exam

☆☆ Super key, Candidate key, Primary key

Explain

⇒ Super key → unique info (Ex. ID) + extra attribute + phone + add

⇒ Candidate key → No extra attributes

NID, passport, ID no

Can be one or more than one attribute

Without extraneous attributes

citizen টার জন্য primary key NID

⇒ Primary key : subset of candidate key.

Candidate key is a subset of super key

☆☆ Exam 6 marks

Candidate key তে name থাকলে তার

coz id is enough for identification. No extra attribute required

7.7.26

Tenary

Convert to binary

Design issues

⇒ attribute এর কিভাবে প্রয়োজনীয় attribute থাকবে

প্রয়োজন attribute টা কে entity বানিয়ে

রেখা হয়

⇒ Relation এর কোন attribute কাউকে people may get confuse যে relation টা কে entity বানিয়ে ফেলব কিনা

Weak entity

⇒ An entity set that has no primary key

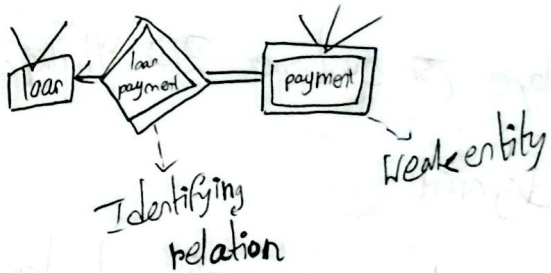
⇒ Bkash এ transaction non persistent that is not primary key

2nd time transaction এ গেলে আগের transaction id so transaction id is not primary key

⇒ Weak entity set যার আগে আর existence dependent is called (identifying entity set) → strong

Ex: Transaction id is dependent on customer id

⇒ Weak entity set relation with strong entity is called Identifying relationship



Specialization

entity set কে কোনো special characteristics দিয়ে আলাদা করতে পারি।

Generalization

⇒ subclass গুলোকে merge করে superclass

Attribute inheritence

superclass এর attribute গুলো subclass inherit করে।

⇒ যে employee সে customer হতে পারবেনা এটা disjoint

⇒ faculty member যেমন একই সাথে student ও হয় It is overlapping

Completeness Constrails

total : সবাই যদি participate করে।

person এর সবাই employee/customer হবেই

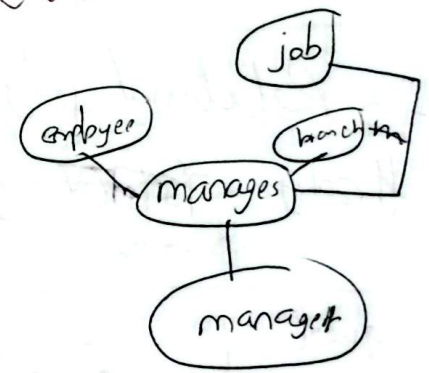
⇒ It's not guranteed. So it is partial

⇒ Superclass থেকে সবাই যদি subclass এ যায় । This is total

Exam

Total, partial, disjoint, overlapped, conditioned define, user define with example

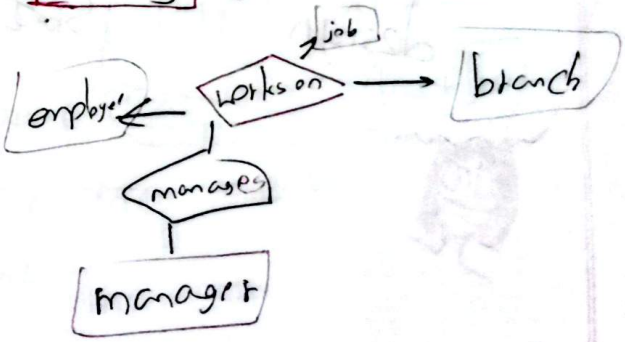
Aggregation



⇒ upper part aggregate

⇒ lower part relation এর connected

এটাকে binary তে বানাব



Avengers Assemble

ER diagram

Representing Entity Sets as Tables

Chp-3 Relational Algebra + theory থেকে 1 quest

quest

Compulsory, may be from design

SQL ~~or number theory~~

DBMS Final

Quest - 1 Compulsory Design like Mid

Sec-B Compulsory : Funtional Dependency Chp 3 & SQL Chp-4

SQL Chp-4

Banking software কে define করে দিয়ে

Quest

- Chp3+ Chp-4

যদি SQL compulsory তে না হয়

Chp-3: View, Rename, Set Divisn (Proof)

Chp-4: Clause কে নিয়ে describe

pure SQL

Must learn SQL queries

- except, view, where with clause
- delete, drop, update, after diff
- create table create view এ মার্কস
- diff একটু virtual
- Which are dml queries which are ddl
- database এ গিয়ে change করে কোন queries

Complex query মুখোমুখি থাকবে

Transaction
Chp-14

Database এ Transaction কাজ করতে হবে
Acc A থেকে minus Acc B তে
গিয়ে plus

ODBC, JDBC / Imbedded → এসব পড়ার দরকার নাই

CT-2 Next week on Chp- 3, 4, 6

Chp-6 Trigger Imp. Assert Imp
Explain with example

Project Database এ Trigger Assertion
Referential integrity করতে হবে

Mahbub Sir | 9th week | Tuesday | 18.8.26

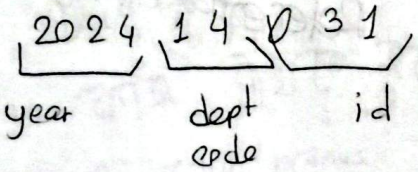
Chp-7: Relational Database Design

Compulsory section of sec-B from this chp or chp-3,4 (mix)

first normal form

domains of all attributes of a relational schema R are atomic
↳ conceptual division is not possible

Example: id = 20241031 it may seem atomic but it can be broken



So atomic কিনা it depends on perspective

⇒ যদি attribute of a table যদি atomic হয় তাহলে table টা first normal form

atomic না হলে কী problem :

Let a staff has id of 005 and posted in CE dept, if the staff is transffered to CSE dept then his id should be CSE 105 but id is primary key, the initial one can't be changed. So non atomic হলে এই problem টা হয় (given in slide)

pk ?? { CE 005, CSE 105 }

problem

Pitfalls in Relational database

1) Repeatation
info repeat হলে পরে যখন update করা হয় সবগুলো update না হলে data extraction a problem

2) Inability to represent certain infor.

Design goals

Redundancy

Null values

other কেউ loan নেই নাই
or, loan নিয়েছে data টা entry হয় নি

Decomposition

redundancy কমাতে decompose

⇒ decompose করার সময় keep 2 things in mind

1) যদি union করি তখন সবগুলো attribute কে খুঁজে পায়
যদি কিছু attri. missing হয় then decomposition হয়
Example
৪ টা attri কে decompose করে 5 আর 3 বানাম, যেকোনো একটা loss হলে decomposition unsuccessful, data loss

⇒ Table R is decomposed into R_1, R_2
যদি $R_1 \cup R_2$ করি তখন যেন R কে খুঁজে পায়

attribute যেন হারিয়ে না যায়

Tuple যেন ফেরত পায়

2) Decompose করার পরে যদি natural join করি তাহলে যাকে decompose করলাম তাকে ফেরত পাব

কোনো tuple যদি হারিয়ে যায় it means

⇒ কোনো info database এ ছিল না যেটা entry হয়ে গেছে (generate)

⇒ কোনো info database এ ছিল যেটা এখন নেই

দুই ক্ষেত্রেই it is a loss

A	B
α	1
α	2
β	1

T

decompose →

A
α
β

T_1

B
1
2

T_2

এখন যদি join করি

$T_A \bowtie T_B \Rightarrow$

A	B
α	1
α	2
β	1
β	2

New generate হয়ে গেছে it is a loss

So decomposition টি lossless হয় নি

Exam

- R কে decompose করা হয়েছে R_1, R_2, R_3
- Check if this decomposition is lossless or not (Proof টি আছে slide এ)

Goal – Devise a theory

Our theory is based on

1) Functional dependencies → Exam এ আসবেই
2) Multivalued "

Functional dependencies

$\alpha \rightarrow \beta$

R = full table

α implies β it means β is functionally dependent on α

α is subset of R	$\alpha \subseteq R$
β " " "	$\beta \subseteq R$

β কোনো জায়গায় যদি α র value বসে আছি সেখানে β র value auto চলে আসবে

Left টা কে আনলে Right টা auto আসবে

Example: Roll no → CGPA

7 → 3.9

যখন roll no. টাকে cgpa র চলে আসবে. Whenever roll no. is refered, CGPA will come. So this is a constraint

⇒ Roll no. → GPA it is not implied coz অন্যান্য পর্যন্ত 48 টা semester এর GPA আছে, GPA is not functionally dependent on roll no.

⇒ Also ~~Roll~~ GPA → Roll no. this notation is incorrect. Because same CGPA র অনেক roll no. থাকতে পারে

Superkey: K is a superkey যদি এই info দিয়ে uniquely identify করতে পারি একটা row, tuple কে

Table, R - (A, B, C, D, E)

Superkey, K = A, B, C → A, B, C কে দিলে R table কে uniquely identify করতে পারি.

So functional dependency হবে

$K \rightarrow R$ (K implies R)

Candidate key: K is a candidate key when there is no subset α in K

$\alpha \subset K$, $\alpha = A, B$ candidate key

for no $\alpha \subset K$, $\alpha \rightarrow R$

যদি কোনো subset হয় α, β

$\alpha \subset K$, $\beta \subset K$

$\alpha \rightarrow R$ $\beta \rightarrow R$

তাহলে K কে superkey বলবো, candidate key বলতে পারবনা

Primary Key: pk → Relation

A → R

A implies R. যেমন, A কে দিয়ে আমরা পুরো একটা info পেয়ে যাবো.

pk → Relation A → (A, B, C, D, E)

A → R (A is pk)

Candidate Key

$K = AB$

$R = (A, B, C, D, E)$

$K \rightarrow R$

$AB \rightarrow R$ (roll no)

$\alpha = B$

$B \rightarrow R$ (regi no)

যদি α র পরে $\beta \rightarrow R$ হয় তাহলে এটা Candidate Key হবেনা

Use of functional dependencies

- r satisfies F
- F holds on R

⇒ Remember these 2 points

→ particular সময়ে satisfy করতে পারে

→ all time satisfy করবে

⇒ loan no → amount
loan no → branch-name
} Holds always satisfy করবে

⇒ loan-number → customer_name } satisfy always satisfy করবেনা

⇒ Functional depency is trivial

$\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

loan number → cust-name

Roll no → CGPA not CGPA → roll no

☆ $\alpha \rightarrow \beta$ doesn't mean $\beta \rightarrow \alpha$ ☆ Exam এ ভুল হয়

Left side আর Right side swap করা যাবেনা

Closure of a set of F.D

If $A \to B$ and $B \to C$ then we can infer that $A \to C$

$R = (A, B, C, D, E)$

$F = \{ A \to B,\ B \to C,\ AD \to E,\ C \to D \}$ ⇒ এই বাইরেও functional dependency থাকতে পারে

যদি functional dependency F হয় then

$F^+ = F^+ + f_1\ f_2\ f_3$

F^+ closure — F^+ যেটা given — $f_1\ f_2\ f_3$ given যা দিল আর বাইরে আরও অনেক functional dependency থাকতে পারে

Armstrong's Axioms
6 টি rule memorize

Exam এ rule আসবে ☆☆

⇒ $\beta \subseteq \alpha$ so $\alpha \to \beta$ (reflexivity)

cust_name ⊆ cust_roll_name

⇒ $\alpha \to \beta$ then $\gamma\alpha \to \gamma\beta$ (augmentation) দুইপাশেই augment করতাম

⇒ $\alpha \to \beta$ and $\beta \to \gamma$ then $\alpha \to \gamma$ (transitivity)

These rules are sound and complete

Learn Example Given in slide

Example

$R = (A, B, C, G, H, I)$

$F = \{ A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H \}$

Given

$A \to C$

$AG \to CG$ (Augmenting)

Given $CG \to I$

So, $AG \to I$ (By augmenting)

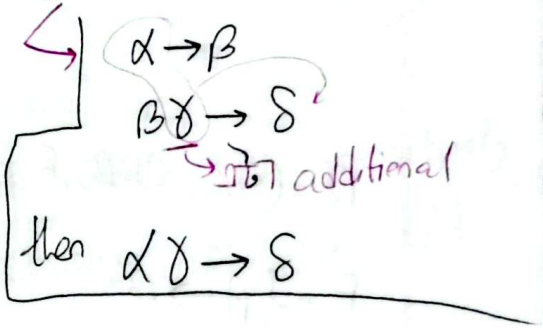
Procedure for computing F^+

Algorithm ex an ১ প্রশ্ন

memorize

প্রত্যেক 3 টা basic rule এর পরে আরও 3 টা additional rule আছে:

union, decomposition, pseudotransitivity



$\alpha\beta \to \gamma$
$\alpha \to \gamma$
$\beta \to \gamma$

No
This is wrong

$\alpha \to \beta$
$\alpha \to \gamma$
$\alpha \to \beta\gamma$ → decompose করলে
$\alpha \to \beta$
$\alpha \to \gamma$

F^+ বানানো a^+ বানানো
Likely to be compulsory question

$R = (A, B, C, D, E)$

$F = \{ A \to B$
$B \to C$
$AD \to E$
$E \to D \}$

$\alpha = (A, B)$

$\alpha^+ = (A, B, \ldots)$

যদি $\alpha^+ = R$ হয়
i.e $\alpha^+ = (A, B, C, D, E)$ হয়
then $\alpha^+ = R$
এটা হবে যদি α
is primary, candidate
or superkey

Closure of attribute set

additional dependency নিয়ে যে set বানাত
তোমা set closure.
Algorithm আছে, reflexity, augmentation খাটিয়ে
change করা কিনা, ~~[illegible]~~ পরে transitivity
করা হয়, যদি change হয় then add
হবে set এ

given in slide the proof

set of attribute = α
table এর column, $R = (A, B, C, D, F)$
$\alpha = (A, B)$
α closure, $\alpha^+ =$
attribute closure
যদি $\alpha^+ = (A, B, C, D, F)$
সব column যদি পায় তাহলে

যদি সব attri. কে দিয়ে তৈরি table R কেই পাওয়া যায়

$\alpha^+ = (A, B, C, D, E) = R$

So, $\alpha^+ \rightarrow R$ ⇒ set of attri. দিয়ে তৈরি row টা কে পাচ্ছি. So α^+ is a superkey

⇒ যদি A' দিয়েই সব বোঝাতে পারি B থাকে extra তাহলে candidate key হবেনা, superkey হবে

example: A = roll no, B = name

roll no টাই enough identify করতে

This is extra

Alg to compute α^+

result = α

if $B \subseteq$ result

then result = result ∪ γ

α = left side of F

$F = \{ A \rightarrow B, \; B \rightarrow C, \; CD \rightarrow E \}$

$\alpha = A, B, C$

$CD \subseteq (A, B, C)$

$B \subseteq$ result

$B \subseteq AB$

$\alpha^+ = (A, B, C)$

A, B is not a superkey so α is not a superkey

যদি $F = \{ A \rightarrow B, \; B \rightarrow C, \; CD \rightarrow E, \; C \rightarrow D \}$

$\alpha = A, B, C, D$

$CD \supseteq CD$

$\alpha^+ = (A, B, C, D, E)$ α is a superkey

Do it

$C \to DE$ থাকলে তাহলে sk কিনা

$C \to D$

$E \to E$

☆ Exam এ F দিবে, α দিবে question

☆ আসবে if it's a sk or ck?

Must আসবে in CT final

Examp-1

$R = (A, B, C, G, H, I)$

$F = \{ A \to B$
$A \to C$
$CG \to H$
$CG \to I$
$B \to H \}$

$(AG)+$

$\alpha = AG$

$\alpha+ = (AG)+ = ABG$
$= ABCG$
$= ABCGH$
$= ABCGHI$

left side subset হলে right side union হলে

$(AG)+ \to R$

$(AG)+$ is a sk

Now check if it's a ck

if $A \to R$ or
$G \to R$ তাহলে ck হবেনা

$\alpha = A$

যদি $\alpha+ \to R$ হয় i.e $A \to R$ হয়

then AG is not a ck

G ছাড়াই যেহেতু হয়ে গেছে so not a ck.

ck check করতে গিয়ে there will be more than one attr. তাহলে আর check

if $\alpha^+ = A$

then $= AB$ ($A \to B$)

$= ABC$ ($A \to C$)

$= ABCH$ ($B \to H$)

CG আসেনি

So A is not a sk. so we cant eliminate G in AG

$\alpha^+ = G$

G is not a superkey

AG is a sk and a ck

Canonical cover

$F^+ = F + \{f_1, f_2, f_3 \dots f_n\}$

$F_c = F - \{f_1, f_2, f_3 \dots f_n\}$ (This is the minimum)

Func Dep কে কমানো যায়

কমায় যেভাবে

1) checking কমাবে
2) efficiency বাড়বে
3) Integrity ti বজায় থাকবে

যদি canonical cover, f_c এর closure বের করি then intermediate একটা stage এ এসে F পাব, আবার F থেকে F closure(f^+) পাব বাড়লে কমলে

$F_c^+ \to F \to F^+$

F_c কে বাড়ালে F

F " " F^+

কী omit করব

Extraneous attri
Extra dependency

$F = \{ A \to B$
$CD \to E$
$E \to F$
$B \to D$
$CD \to I$
$C \to BD \}$

Circle করা গুলো omit করব canonical তে নিবে

Canonical means:
minimum in terms of functional dependency
" " " " attribute

side এ extra attri page এ
মুলতে যদি left side এ extra হয়ে
থাকে " right side এ " "

If $\beta = \underset{\downarrow}{AB}$
let this is extra
$C \to AB$
FD থেকে এটা বাদ হয়ে যাবে
আর add হবে
$C \to B$

Exam এ extra attri কে find out করে
extra FD বাদ দিয়ে canonical cover
বের করা আসবে Exam

Given,

FD = {A → C
AB → C
A → C

A → C

AB → CD

× AB → C

AB → D

A → C

AB এ থেকে B বাদ দিলে
A → C.
আবার A → C already
আছে so C কে বাদ
দিতে পারি
so omit AB → C

যেহেতু AB → D থাকলে
can't eliminate A → D
caz A → D নাই

Given,

F = {A → BC
B → AC
C → AB}

= {A → B
A → C → ১ম বাদ দিতে পারি caz
A → B হলে C → C হলে
A → C আর থাকবে না
B → A
B → C
C → A
C → B}

= {A → B
B → A
B → C
C → A → ১ম বাদ দিতে পারি caz
C → B হলে B → A
হলে C → A আর
থাকবেনা
C → B }

Alternative
can omit
this

= { A → B
B → A
B → C
C → B }

→ Augment করা যায়

= { A → B
B → AC
C → B }

This is a canonical cover

Alternative

C → B এটা আগের page এ ছিল

= { A → B
B → A
B → C
C → A }

এটা avoid করতে পারব

= { A → B
B → A
C → A }

→ This is fc

Testing if it's extra

⇒ যেটাকে extra করছি সেটাকে বাদ দিয়ে যদি closure বের করতে পারি then its confirm that it's extra

Follow example - 1 of Attribute ~~of~~ is Extraneous slide

This is easier

⇒ Do all example

⇒ Why it's important

Canonical cover of functional dependency in dbms

push_back()

Mahbub Sir

27.8.26

⇒ Exam এ normalizat দেওয়া থাকবে

⇒ Example থেকে first Normal, 2nd Normal Schema দেওয়া থাকবে Normalizat করতে হবে

$R = (A, B, C, D, E)$

$F = \{A \to B, B \to C, D \to E\}$

$R_1 = (A, B, C)$

$R_2 = (A, D, E)$, intersect

$R_1 \cap R_2 \to R_1$ OR

$R_1 \cap R_2 \to R_2$

$R_1 \cap R_2 = A$

যদি $A \to R_1$ or $A \to R_2$ হয় তাহলে এটা loss less

Decompose করতে হলে it has to be lossless

Test for lossless join → Learn

⇒ 2NF এর first condit এটা 1NF হতে হবে

⇒ 2NF এ composite attribute হবেনা

⇒ 2NF এ দুইটা ভিন্ন একটা composite key possible

Non prime: candidate key এ নাই এমন attribute

Partial dependency

Student name has dependency with Student id

has no " " Course id

⇒ Fully dependent হলেই 2NF এ যাবে

BCNF

⇒ 2 Conditions (OR condit)

⇒ Calculat দেওয়া হয়

Example

$R = (A, B, C)$ $F = \{A \to B, B \to C\}$ Key = $\{A\}$

Ans

$A^+ = A$

$= AB$

$= ABC$

$A \to R$

1.9.26

⇒ BCNF may be preserved or may be not

Then we go for 3rd normal form

1) Redundancy allowed

2) lossless

3) dependency preserving

3NF এর 3 টা condition এর যেকোনো একটা হলেই হবে

$R = (A, B, C, D, E)$ → for 3rd condition

$\beta - \alpha$.... explanation

$F = \{A \to B$

$B \to C$

$CD \to E$ $\alpha = AB$

$E \to C\}$ $\beta = BC$

$\beta - \alpha$ = β তে আছে, α তে নাই

$= C \to$ এই ans টা হবে LHS এর যেকোনো একটা candidate key র অংশ

LHS এই Candidate Key থাকবে

Now if $\alpha = B$

$\beta - ABC$

$\beta - \alpha = AC$ → Ans এ যদি 2 টা attri থাকে then 'A' একটা ck এর অংশ and 'C' অন্য ck এর অংশ হলেও লিখবে একসাথে হওয়া লাগবেনা

ck1 ck2

অথবা AC দুটোও ck হতে পারে

Problem of 3NF

⇒ Banker schema, bank name repeat

int* a = (int*) arg

What is prime attr

$\alpha = AB$

$\alpha^+ \rightarrow R$

$AB \rightarrow R$

$AB \rightarrow$ sk/Ck

A একটা prime attri

B আরেকটা " "

Non prime attri : ck ছাড়া যে দুটো আছে

id → ck → prime attr

name → depends on id → not ck → non prime

again

dept name → depends on id
→ also depend on dept_id

dept id → depends on id

So dept_name is a non prime attr. which depends on another non prime (dept-id).

so this is transitive. Not applicable in 3NF. Need to be decomposed

BCNF and 3NF same condition or not

Comparison

→ BCNF is subset of 3NF

→ 1st Normal এর subset 2nd normal

Mahboob Sir

Chp-12: Indexing

3.9.26

⇒ Hashing বাদ

⇒ Index authorization is restricted

resources wastage হবে, overhead

pk এর উপর database by def. index file create করে

⇒ Index authorization দেওয়া এমন কোন যেকোনো attribute এর উপর index file create করা

fig 14.5

⇒ Data file unclustered হলে Block of record pointers dense হতে পারে

⇒ Multilevel indexing → static, dynamic

Compulsory

Design, SQL, Chp-7

8.9.26

Static index → defined structure

⇒ overhead হয়ে যায় তবুও periodic maintainance is required

Dynamic index

⇒ insertion, delete এর সময় ই restructure হয়ে যায়। No periodic maintainance.

⇒ auto adjusted

Index file

⇒ 3 টা বিষয়

⇒ যার উপর indexing করছি

⇒ কোন key দিয়ে খুঁজছি

⇒ file টার location

Composite key থাকলে index table

⇒ এই 3 টা column + pointer এর column

⇒ n টা search key হলে index table এর size (n+1)

12.32 → inserting 18 further overflow 19

⇒ Searching time বেড়ে as overhead যত বেশি হয়, বেশি comparison করা লাগে। Overhead বাড়লে tree বাড়ে, length বাড়ে। This is why we need periodic maintainance.

⇒ Restructuring করে overflow area থেকে নিয়ে আসছি

Dynamic এর 4 types যেকোন একটা complete করে আসে final এ

Tree এর structuring, restructuring

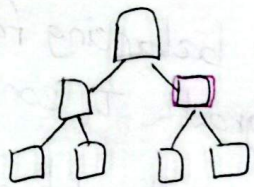
Exam

Binary Tree → limitation is unbalanced হলে searching বেশি যায়

Binary: Max 2 children

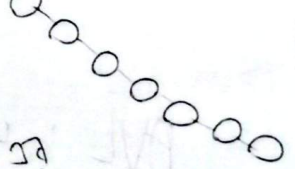
Height যত বড়, searching complexity তত বাড়ে

Balanced



⇒ 7 টা record এর জন্য 2 টা level

Unbalanced হলে



⇒ 7 টা record এর জন্য 6 টা level. অনেকগুলো searching

AVL

⇒ Table এর growth যা হয়.

⇒ $\log_2(N+1) < \text{height} < 1.4404 \log_2(N+2) - 0.328$

⇒ 100 টা record হলে growth

⇒ $6.658211 < \text{height} < 9.282961$ (AVL)

⇒ Height টা restrict এর জন্যই তবে আসছে but in inbalanced binary 100 টা record এর 99 টা পর্যন্ত growth

⇒ Height টা control করা যায় balancing factor দিয়ে এটা দিয়ে AVL tree এর growth টা control করা যায়

⇒ Balancing factor = height (left subtree) − height (right subtree)

BF = {-1, 0, +1}

AVL : unbalanced Binary Tree restructure করে still unbalanced factor -1, +1 accept করে

Multiway

No of child can be more than 2

B-Trees

⇒ Definition দুইটা slide এর মধ্যে

Order (4) হলে 2 to 4

Order (6) হলে 3 to 6

Order (5) " 3 to 5 (2.5 not possible) Ceil

একটা record হলে দুইটা root হবে

দুইটা " " root + 1 child

তিনটা " " root + 2 "

White down the property of B tree and other trees

☆☆ Exam

⇒ M-1 করলে unbalanced factor টা maintain করে

⇒ Terminal node same distance from root

$K = M/2 < \text{to } M$

$\Rightarrow \frac{M}{2} \leq K \leq M$

K = no. of children

Record $k-1$ Non terminal

Terminal $\rightarrow k = M/2 - 1$ at least

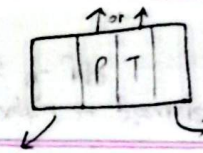
Record সংখ্যা, $k = M - 1$

Order (3) হলে record সংখ্যা

$k = M/2 - 1$

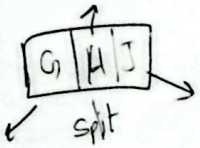
$K = M - 1$

⇒ এর পরের topic থেকে Exam এ must একটা questth থাকবে



Too big condition

⇒ Too big হলে restructuring হয়



⇒ বিভাজ্য হলে ভাঙা হলে problem

Too small condition

Merge with sibling

⇒ Order বেশি হলে searching time কমে

Order (5) হলে

Max Children $= K = 5$

Min " $= \frac{k}{2} = 5/2 =$

Highest Record $= K = M - 1 =$

Lowest Record $= M/2 - 1 =$

Biasing দিয়ে রাখা থাকতে পারে in delet
Biased to left / right

Insert and delet এসবে related too small / too big

no of proc. sch. long awk? diff -h ? chmod +x fi. chmod + rwx same? chmod 777 all permiss read, write, acc.

B^+, B^*, AVL tree property then insertion deletion, growth

Exam

A, R, U → insert key

Root এ max record, $k \leq m$

Order 4 হলে min record $\geq \frac{m}{2} - 1$

$= \frac{4}{2} - 1$

$= 1$

max record = $m = 4$

3 টা key arbitrarily insert করলে no branching

Comparison Tree variant

Exam

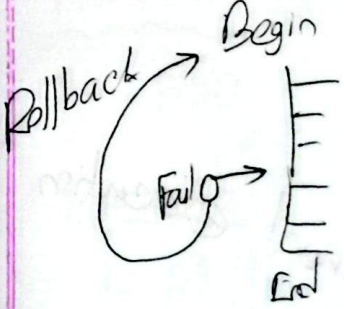
Transit Chp-15 13th week Tuesday 15.9.26

Atomicity: Transact either সবগুলা execute হবে or কোনোটাই না

Concurrent access time ১ correctness নষ্ট হওয়ার chance Consistency need to be maintained

Durability → RAM ১ থাকলে হারিয়ে, permanent memory তে থাকবে updated value টা

⇒ durability ensure করলে successful



⇒ Diff. betn partially committed and committed

Ans: Durability

awk

f.(1 2 3)

PC, Register set, Stack — Thread

PID, Process state — PCB

Void বাদে যে কোনো datatype নিতে পারে int, float.

15.16

A = 1000

A → 950

B = 500

B → 550

Now in T_2

temp = 95

A = 855

B = 645

After Trans = 1500

Before T = After T

Serially execute হচ্ছে

4th.

&P1

Pthread_create (&P1, NULL, func, &arg.)

15.17

A = 950

= 95

A = 855

B = 50

= 550

550 + 95 = 645

Concurrent execution

15.18

A = 1000

-950

write

In T_2 → (write করিয়াছে তাই 950 set হয়ে নাই)

A = 1000

temp = 100

A = ~~1000~~ ~~950~~ 900

B = 500

Now in T_1 → write (A) হয়ে গেছে, তাই T_2 এ read করেনাই so A = 950, in this case Thread কত A = 900 হত

B = 550

Now in T_2

B এর value read করেনাই so 550 হয়নাই

B = 500 ↗

B = 500 + temp (100) = 600

At the end, B = 600, A = 950

Total = 1550

15.19 sequence

⇒ T_1 to DB: A then T_1 → write (A) → A=50

⇒ T_2 to DB: A then read(A) then write(A)
initial val new (A)
then
A = 150

এভাবে schedule বানানো যাবেনা, Initial val read করবে → এটা problem

---

⇒ Serial vs Serializability
↓
concurrent হবে execut" but effect টা serial এর মতোন হবে

⇒ Conflict serializability } Exam এ ques হবে
View "

In exam এটির schedule → simplify read/write বাদে বাকি সব ignore

⇒ যদি write এটে same data তে → conflict
→ conflict এটবে না no swap

15.21

⇒ Conflict হবে কিনা 4 টা condit" → Exam

⇒ Show if it's serializable ← 15.17
schedule দিয়ে এরম ask করবে
just read, write consider করবি
then non conflicting কে swap করবে

15.18

conflicting এ swap করেছি তাই problem হয়েছে

Exam:
ACID property
States of transact"
Rollback? Commit? Partially commit,
Durability ensure how
Conflict, View এটো main quest"
final এ (schedule দিয়ে যাবে)

(Mahboob Sir) 17.9.26

Conflict Serializability

⇒ Conflict equivalent

View serializability

⇒ 15:30 → এটা Exam এ আসবে ☆

⇒ schedule দিয়ে বলবে View serializable নাকি conflict ☆

example given in slide

15:30 3 টা conditions

AND condition

pg - 15:31

Not view equiv

⇒ read (A) in T_1 but পরেরটা fig এ

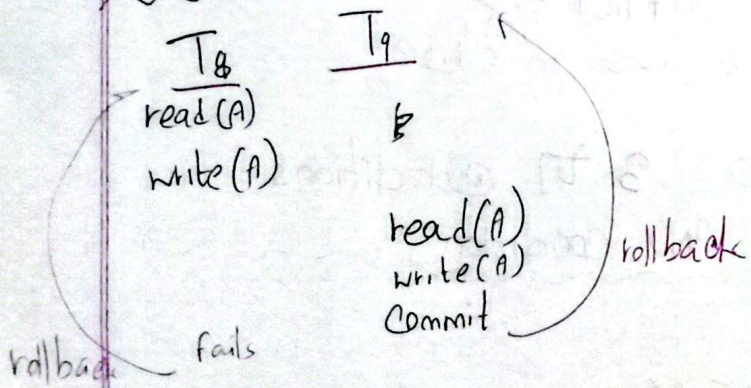
read(A) in T_2 → condⁿ-1 violate

Exam

- Explain every conflict se.. schedule is also view ser. but every view ser is not conflict seri..

Blind write ??

Irrecoverable



⇒ T8 rollback করবে কেন T9 commit করে দিয়েছে T8 শেষ হওয়ার আগেই তাই T8 fail করবে, T8 এর value টাই T9 পড়ে। So T8 fail করলে, T9 ও fail

Cascading rollback

⇒ T8 rollback করলে T9 এর ও rollback করা উচিত

⇒ cascading is not desirable

⇒ cycle create হলে কোনো একজনের failure এর জন্য deadlock create হবে

Set 1 — Compulsory — Set A
DBMS mid

Set 2

Compulsory — Set B — CT-3 → Chp 7 dependencies Change করে দিবে + Canonical cover + Theory based → Algorithm → F-closure, F^+, F_c, α^+

Concurrency ক্লাস থেকে যতটা হয়েছে in whatsapp
14th week Tuesday 22.9.25

Conflicting Non conflicting swap
final এ আসবে

flex calc.l
bison calc.y
gcc calc.tab.c lex.yy.c -

Two phase, Time stamp protocol → Exam

Tree based protocol
Validat phase → correctness বোঝা
schedule produced → ✗
multiple gran, hierarchy, lock mode, mat lock mode, multiversion, ordering, locking → XX
deadlock → ✗

Date:

deadlock prevent → ✗

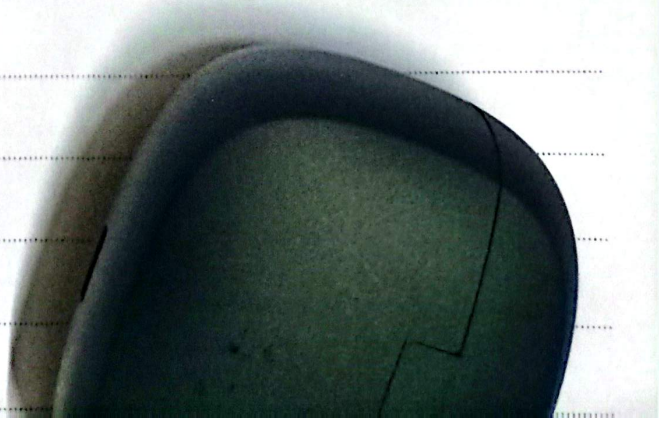
V.V.I Wound-wait

deadlock detect → cycle হলে deadlock

graph এ plot করে দেখার cycle কিনা

deadlock recovery → ✗

starvat^n happens....



CamScanner